

Lab Guide — Lakeflow Connect & Orchestration

Work top to bottom. Everything lives in **your** schema inside the `dev-sh-training` catalog. Set the `schema` widget to your own name on every notebook before you run it.

- Catalog (fixed): `dev-sh-training`
- Your schema: whatever you set in `00_setup` (e.g. `jane_doe`)
- Landing volume: `/Volumes/dev-sh-training/<you>/landing/`

⚠ How to use every notebook in this lab — read first. The widgets (`schema`, `scenario`, ...) are defined in the **first cell**. You must **run that first cell** before anything else so the widget boxes appear at the top of the notebook. Then fill in your `schema` (and any other options) and run the remaining cells (or **Run all**). If you skip the first cell, the widgets never appear, and the notebook falls back to the placeholder default and stops with an error.

The value you type is substituted everywhere automatically — via `${schema}` in SQL notebooks and via the widget in Python notebooks.

Setup (once)

1. Add this repo as a **Git folder** (Workspace → Create → Git folder, URL `https://github.com/realboom/lakeflowconnect`).
2. Open `notebooks/00_setup`, **run the first cell** so the `schema` widget appears, set it to your name, then **Run all**. You now have a `schema` and a `landing` volume with `filedrop/` and `sftp/` subfolders.

Part A — COPY INTO (batch file ingestion)

Concept: `COPY INTO` loads *new* files from a folder into a table and is **idempotent** — rerun it and already-loaded files are skipped. Simple, batch, great for “a file shows up, pull it in.”

1. Open `generate_data`, set `schema`, `scenario` **Part A - file drop (COPY INTO)**, `num_rows` = 25, **Run all**. → writes a comma CSV to `landing/filedrop/incoming/`.
2. Open `part_a_copy_into`, set `schema`, **Run all**. → creates `bronze_file_eligibility` and loads the file.
3. **Run the COPY INTO cell again.** It loads **0 new rows** — that’s idempotency (it remembers what it already ingested).

4. Generate another Part A file (step 1 again) and re-run COPY INTO → only the **new** file is loaded.

✅ Takeaway: COPY INTO = the simplest incremental batch ingest.

Part B — Auto Loader (“SFTP” source, mimicked on a Volume)

Concept: A vendor normally drops files on **SFTP**. We mimic that by landing pipe-delimited files in `landing/sftp/incoming/` and let **Auto Loader** (`cloudFiles`) ingest them incrementally. Auto Loader keeps a **checkpoint** of what it has processed and is built for continuously-arriving files.

1. Open `generate_data`, scenario **Part B – SFTP source (Auto Loader)**, `num_rows` = 25, **Run all.** → writes a **pipe-delimited** CSV to `landing/sftp/incoming/`.
2. Open `part_b_autoloader`, set `schema`, **Run all.** → creates `bronze_sftp_eligibility` and ingests the file. The last cells show the data.
3. Generate another Part B file and re-run Part B → only the **new** file is picked up (checkpoint).

✅ Takeaway: Auto Loader = incremental, checkpointed ingestion — the standard for flat-file / “SFTP” feeds. Contrast with Part A: COPY INTO is batch SQL; Auto Loader is a streaming source with schema evolution + rescue (next).

Part C — Auto Loader resilience

Same `bronze_sftp_eligibility` pipeline from Part B. We show the two ways Auto Loader absorbs drift. **Do rescued data first** (it runs on the original schema), then schema evolution (it permanently adds a column).

C1 — Rescued data (bad value, original schema)

1. `generate_data` → scenario **Part C – rescued data (SFTP)** → **Run all.** Half the rows have a bad `eff_date` of `N0T-A-DATE`. (`eff_date` is typed `DATE` via `schemaHints`.)
2. `part_b_autoloader` → **Run all.** The bad values can’t cast to `DATE`, so Auto Loader **rescues** them: the row still ingests with `eff_date = NULL`, and the original value is parked in `_rescued_data`.
3. The last cell shows the rescued rows.

💡 A malformed value didn’t crash the pipeline or get silently dropped — it’s captured for audit/backfill.

C2 — Schema evolution (a new column)

1. `generate_data` → scenario **Part C - schema evolution (SFTP)** → **Run all**. The file has a new `risk_tier` column.
2. `part_b_autoloader` → **Run all**. Auto Loader detects `risk_tier`. In a notebook it **stops once** to record the new column — if you see an “unknown field”/schema-change message, **just run the cell again** and it succeeds, with `risk_tier` added to the table (existing rows `NULL`).
3. `SELECT risk_tier, count(*) ... GROUP BY risk_tier` to confirm.

💡 A new field appeared and the table evolved on its own. (In Part E a task **retry** makes this one-run automatic.)

Part E — Orchestration (Lakeflow Pipelines + Jobs)

Now we orchestrate the eligibility data you just created. You’ll build **one pipeline** and a few **jobs** by hand.

E1 — The silver pipeline (Spark Declarative Pipeline)

Create a pipeline whose source is `notebooks/silver_eligibility`. 1. **Pipelines** → **Create pipeline** (ETL / Lakeflow Declarative Pipeline). 2. **Source code**: select `notebooks/silver_eligibility` from your Git folder. 3. **Destination**: Catalog = `dev-sh-training`, Schema = **your** schema. 4. **Configuration**: add a key `schema` with value = **your** schema (the source reads `${schema}` to find your bronze table). 5. **Serverless** compute. Create, then **Run** the pipeline once. → builds `silver_eligibility` (streaming table) from `bronze_sftp_eligibility`.

E2 — Table-update trigger (event-driven)

Make the silver pipeline run **automatically when new bronze data lands**. 1. **Jobs** → **Create job**.

Add one task: - Task name `refresh_silver`, Type **Pipeline**, select your silver pipeline. 2. **Add a trigger** on the job → type **Table update** → table `dev-sh-training`.

`<you>.bronze_sftp_eligibility`, condition **ANY_UPDATED**. (Set min time between triggers = 60s.) 3. Test it: run `generate_data` (Part B) then `part_b_autoloader` so new rows land in bronze. Within ~a minute the job fires on its own (run origin: *“Triggered by table update”*) and refreshes silver.

💡 The trigger baselines when you create it and fires on **new** commits — so make a fresh bronze load after you set it up.

E3 — The DAG: gold → report → notify (with a conditional branch)

Create a multi-task job. 1. **Jobs** → **Create job**, name it `eligibility_gold`. 2. Task **gold** — Type Notebook → `notebooks/gold_eligibility_summary`, param `schema` = your schema. Under **Advanced** → **Retries**, set **Max retries** = 2. 3. Task **report** — Type **Notebook** →

`notebooks/eligibility_report`, param `schema` = your schema. **Depends on** `gold`. 4. Task **notify_pass** — Notebook `notebooks/notify`, param `result` = `pass`. Depends on `report`. **Run if** = **All succeeded**. 5. Task **notify_fail** — Notebook `notebooks/notify`, param `result` = `fail`. Depends on `report`. **Run if** = **At least one failed**. 6. **Run** the job → `gold` → `report` → `notify_pass` (and `notify_fail` is skipped). Green path.

E4 — Repair / partial re-run (recover from a schema change)

A realistic failure: someone renames a column upstream and the downstream report breaks. 1. Open `gold_eligibility_summary` and rename the output column `member_count` → `enrolled_members`, save. 2. **Run** `eligibility_gold`. → `gold` **succeeds** (table rebuilt with the new name), but `report` **fails**: `UNRESOLVED_COLUMN ... member_count ... Did you mean 'enrolled_members'?`, and `notify_fail` runs. 3. **Fix** `eligibility_report`: change `member_count` → `enrolled_members`, save. 4. On the failed run click **Repair run**. → `gold` **is skipped** (already built), only `report` + `notify_pass` re-run → success.

💡 You don't rerun the expensive upstream that already succeeded — fix and resume from the failure. (To reset for a repeat: revert both files back to `member_count`.)

E5 — For each (fan-out + concurrency)

Run a task once per line of business, in parallel. 1. **Jobs** → **Create job** `eligibility_by_lob`. Add a **For each** task. - **Inputs**: `["Commercial","Medicaid","Medicare Advantage","Individual"]` - **Concurrency**: 4 2. **Add a task to loop over** → Notebook `notebooks/for_each_lob`, params `schema` = your schema and `lob = {{input}}`. 3. **Run** → four iterations run in parallel, each building `gold_elig_<lob>`.

Concurrency FAQ (you'll be asked): concurrency is 1–100 (default 1); effective parallelism = `min(concurrency, # inputs)`; the workspace cap is **2,000 concurrent task runs**. Size it to what your downstream can handle, not the max. See the instructor runbook for the full FAQ.

E6 — Retries

Notice the **Max retries = 2** you set on the `gold` task. Retries heal transient failures — and they're exactly what makes the Part C **schema-evolution** handshake complete in a single automated run (Auto Loader stops once to record the new column; the retry picks it up). Put a retry on any task that ingests evolving files.

Clean up / start over

Run `notebooks/reset` (set your `schema`) to drop the lab tables and clear the landing folders. Your schema and volume stay. Re-run `generate_data` to begin again.